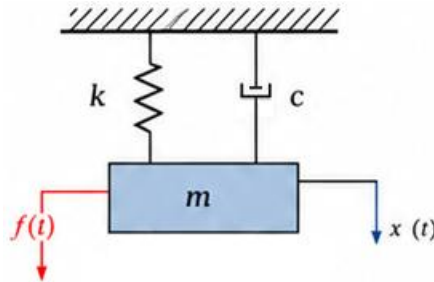


**UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA DE COLOMBIA
INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
SIMULACIÓN DE COMPUTADORES**

TALLER FINAL

INTRODUCCIÓN AL MODELADO Y DESARROLLO DE UN SIMULADOR EN BASE A FUNCIONES DE TRANSFERENCIA

1. MODELO MATEMÁTICO



Segunda ley de Newton

$$\sum F = m \cdot a(t)$$

$$m \cdot \ddot{x}(t) = k(0 - x(t)) + c(0 - \dot{x}(t)) + f(t)$$

$$m \cdot \ddot{x}(t) = k(0 - x(t)) + c(0 - \dot{x}(t)) + f(t)$$

$$m \cdot \ddot{x}(t) + k \cdot x(t) + c \cdot \dot{x}(t) = f(t)$$

Aplicando transformada de Laplace

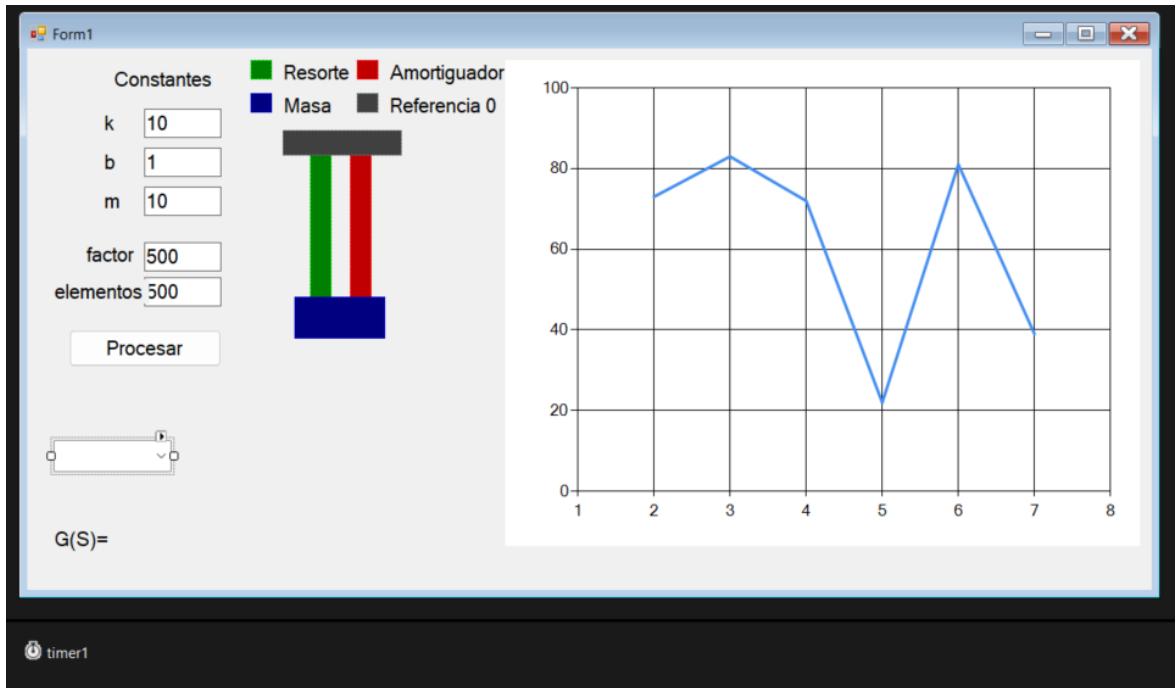
$$X(s)[ms^2 + cs + k] = F(s)$$

Organizando la ecuación se determina la función de transferencia

$$\frac{X(s)}{F(s)} = \frac{1}{ms^2 + cs + k}$$

2. DESARROLLO DEL SIMULADOR

(Anexo en el aula virtual)



```
using Octave.NET;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace MasaResorte
{
    public partial class Form1 : Form
    {
        int coordenadaInicialY;
        int tamañoInicialKB;
        int j = 0;
        double[] vectorY = new double[3];
        double[] vectorT = new double[3];
        public Form1()
        {
            InitializeComponent();
        }
        private void Form1_Load(object sender, EventArgs e)
        {
            OctaveContext.OctaveSettings.OctaveCliPath =
                @"D:\Software\octave-11.1.0-w64\octave-11.1.0-w64\mingw64\bin\octave-cli.exe";
            coordenadaInicialY = pictureBoxM.Location.Y;
            tamañoInicialKB = pictureBoxR.Height;
        }
        private void button1_Click(object sender, EventArgs e)
        {
            var Octave = new OctaveContext();
            string instrucciones = "clc;"
                + "clear;"
                + "pkg load control;"
                + "s=tf('s');"
                + "k=" + textBoxK.Text + ";"
                + "b=" + textBoxB.Text + ";";
        }
    }
}
```

```

+ "m=" + textBoxM.Text + ";";
+ "G=1/(m*s^2+b*s+k);";
+ "[y,t]=step(G);";
+ "c=length(y);";
+ "tiempo=t(c);";
+ "[y,t]=step(G,tiempo,tiempo/"+ textBoxElementos.Text + ");";

Octave.Execute(instrucciones);
Array.Resize(ref vectorY, Convert.ToInt32(textBoxElementos.Text));
Array.Resize(ref vectorT, Convert.ToInt32(textBoxElementos.Text));
vectorY = Octave.Execute("y").AsVector();
vectorT = Octave.Execute("t").AsVector();
for (int i = 0; i <= Convert.ToInt32(textBoxElementos.Text); i++)
{
    chart1.Series[0].Points.AddXY(vectorT[i], vectorY[i]);
}
timer1.Enabled = true;
}
private void timer1_Tick(object sender, EventArgs e)
{
    pictureBoxM.Location = new Point(pictureBoxM.Location.X,
        (int)(coordenadaInicialY + vectorY[j] * Convert.ToInt16(textBoxFactor.Text)));
    pictureBoxR.Height = (int)(tamañoInicialKB +
        vectorY[j] * Convert.ToInt16(textBoxFactor.Text));
    pictureBoxB.Height = pictureBoxR.Height;
    j++;
    if (j==vectorY.Length)
    {
        timer1.Enabled=false;
    }
}
}
}
}

```

NUGET: OCTAVE.NET

<https://www.nuget.org/packages/Octave.NET>

Octave.NET es un paquete de NuGet para usar GNU Octave desde aplicaciones .NET, por ejemplo en C#. Sirve como puente entre un programa desarrollado con el IDE Visual Studio y Octave para ejecutar cálculos numéricos, operaciones matriciales y scripts con sintaxis de Octave.

En términos prácticos:

- NuGet es el gestor de paquetes de .NET.
- GNU Octave es un entorno de cálculo científico compatible con muchos scripts de MATLAB.
- Octave.NET permite conectar ambos mundos desde C# o proyectos .NET.

Su uso típico sería cuando se requiere que una aplicación en C# aproveche rutinas matemáticas ya escritas en Octave, sin reimplementarlas desde cero.

TALLER: MODELADO MATEMÁTICO Y SIMULACIÓN COMPUTACIONAL DE UN SISTEMA MASA-RESORTE-AMORTIGUADOR DE TRES MASAS

Analizar, modelar y simular computacionalmente un sistema mecánico de tres masas acopladas mediante resortes y amortiguadores, obteniendo sus funciones de transferencia y desarrollando una aplicación que permita estudiar su comportamiento dinámico ante diferentes combinaciones paramétricas.

Sistema a analizar

Se trabajará con un sistema vertical compuesto por:

- Tres masas:

$$m_1, m_2, m_3$$

- Cuatro resortes:

$$k_1, k_2, k_3, k_4$$

- Cuatro amortiguadores:

$$c_1, c_2, c_3, c_4$$

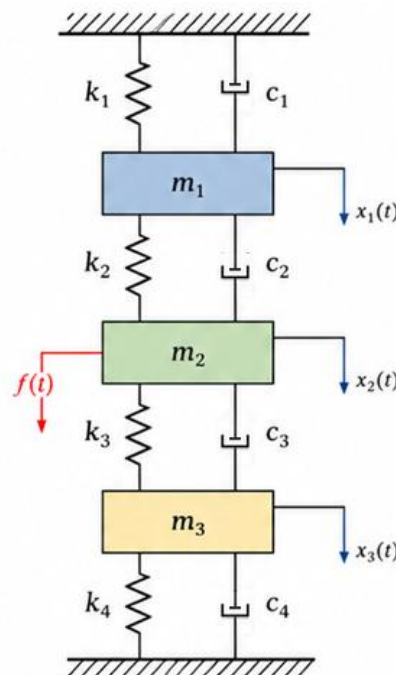
- Una fuerza externa:

$$F(t)$$

aplicada sobre la masa determinada en la evaluación del segundo corte.

- Tres desplazamientos:

$$x_1(t), x_2(t), x_3(t)$$





Realice el análisis matemático y determine las funciones de transferencia para cada masa, documente el proceso y presente el desarrollo del análisis matemático en un archivo PDF, implemente un editor de formulas.

$$G_1(s) = \frac{X_1(s)}{F(s)}$$

$$G_2(s) = \frac{X_2(s)}{F(s)}$$

$$G_3(s) = \frac{X_3(s)}{F(s)}$$

Desarrollo del simulador

Entorno de implementación: El simulador deberá construirse en Visual Studio y se podrá utilizar una de las siguientes tecnologías:

- .NET Framework 4.7.2
- .NET Framework 4.8
- .NET 8
- .NET 10

Configuración paramétrica del sistema

El programa deberá permitir al usuario definir rangos de valores para un tipo de cada constante del sistema:

$$k_1, k_2, k_3, k_4$$

$$c_1, c_2, c_3, c_4$$

$$m_1, m_2, m_3$$

Es decir, que deberá elegir una constante k , una constante c y una constante m

Para cada parámetro elegido se deberán ingresar tres datos:

1. Límite inferior.
2. Límite superior.
3. Tamaño del paso.

Ejemplo

Para:

$$k_1: \text{mínimo} = 1, \text{máximo} = 2, \text{paso} = 0.2$$

el sistema deberá generar internamente los valores:

$$1, 1.2, 1.4, 1.6, 1.8, 2$$

Generación de combinaciones e iteraciones

El simulador deberá construir todas las combinaciones posibles entre los rangos de valores definidos.

Como se eligen sólo 3 constantes para asignar rangos, el número total de iteraciones se calculará como:

$$N = n_k n_c n_m$$

donde cada n corresponde a la cantidad de valores generados para un parámetro.

Requisitos de control

Antes de iniciar la simulación, la aplicación deberá mostrar:

- Número total de combinaciones.
- Número total de simulaciones a ejecutar.

Resultados gráficos

Para cada iteración se deberán generar las siguientes gráficas:

Respuesta al paso

- $x_1(t)$
- $x_2(t)$
- $x_3(t)$

Respuesta al impulso

- $x_1(t)$
- $x_2(t)$
- $x_3(t)$

El simulador deberá implementar una estrategia que permita organizar las gráficas generadas para todas las iteraciones de la simulación, donde se muestren los parámetros de simulación (valores de las constantes) y las gráficas de las respuestas al paso e impulso para cada masa.

Una primera alternativa consiste en presentar los resultados en un DataGridView, incluyendo una columna de tipo imagen en la que se visualice una miniatura de la gráfica asociada a cada iteración. Para ello, el estudiante podrá generar la gráfica en un control Chart, capturarla por código como imagen y almacenarla o mostrarla dentro de la tabla junto con los parámetros y métricas correspondientes.

Una segunda alternativa consiste en crear dinámicamente los controles Chart mediante código y agregarlos a un contenedor visual, por ejemplo, un Panel o FlowLayoutPanel, habilitado con barra de desplazamiento vertical. De esta manera, cada iteración podrá mostrar sus gráficas de respuesta al paso y al impulso dentro de una interfaz desplazable, permitiendo recorrer de forma ordenada todos los resultados generados sin depender de una única gráfica reutilizable.

El grupo de trabajo puede elegir cualquier otra estrategia para la presentación de las gráficas de todas las iteraciones.

Métricas

Para cada iteración, el simulador deberá calcular y mostrar:

Máxima elongación o desplazamiento absoluto para cada masa.

Estas métricas se deberán obtener para:

- Respuesta al paso.

- Respuesta al impulso.

Tiempo de simulación

El programa deberá registrar:

- Tiempo de simulación de cada iteración y para cada respuesta.

Animación del sistema

El simulador deberá incluir una animación gráfica del sistema masa–resorte–amortiguador.

La animación deberá representar el comportamiento correspondiente a:

- La última iteración ejecutada y deberá representar visualmente:
- Masa m_1
- Masa m_2
- Masa m_3
- Resortes.
- Amortiguadores.
- Soportes superior e inferior.

La animación deberá:

1. Mostrar el desplazamiento vertical relativo de cada masa.
2. Reproducirse secuencialmente según el vector temporal.
3. Corresponder a una de las dos respuestas dinámicas válidas:
 - Respuesta al paso
 - Respuesta al impulso

Incluir:

- Reproducir.
- Pausar.
- Reiniciar.
- Indicador del tiempo actual.
- Selección entre animar respuesta al paso o respuesta al impulso.

Validaciones mínimas

La aplicación deberá verificar al menos que:

- Los pasos sean mayores que cero.
- El límite superior no sea menor que el límite inferior.
- Las masas sean positivas.
- Los coeficientes de rigidez sean positivos.
- Los coeficientes de amortiguamiento no sean negativos.
- No se inicie una simulación si faltan datos esenciales.
- Un textbox para escribir la ruta del cli de octave (ya que este variará con el cambio de ejecución en equipo).

Entregables

Cada grupo deberá subir al aula virtual una carpeta comprimida con:

1. Informe en PDF que incluya:
 - Desarrollo matemático.
 - Funciones de transferencia.

- Tecnología seleccionada.
 - Capturas de interfaz.
2. Carpeta del proyecto completo.

RÚBRICA DE EVALUACIÓN		
Criterio	Descripción	Puntaje
Desarrollo del simulador	Interfaz funcional, ingreso de rangos y generación de combinaciones.	15
Gráficas de simulación	Visualización clara de respuestas para las tres masas.	15
Métricas obligatorias	Máximos desplazamientos y tiempos de simulación por iteración.	15
Animación del sistema	Representación coherente del movimiento dinámico de las masas.	10
Organización y calidad del software	Validaciones, estructura del código, claridad y usabilidad.	20
Modelo matemático e informe técnico	Calidad de la documentación y evidencias entregadas.	25
	Diagramas, ecuaciones y paso correcto al dominio de Laplace.	
	Obtención correcta de $G_1(s)$, $G_2(s)$ y $G_3(s)$.	
Total		100

Fecha de entrega: viernes 5 de junio 6pm
Copia en código, nota igual a cero

Está permitido implementar herramientas de IA y generación de código.